

Lunara

AI 应用案例研究 · Women's Health Agent

Frannie · 2026.05 · github.com/hogomex393-svg/Lunara

1. 项目背景与目标

Lunara 起初是一个用 React 写的女性健康追踪应用 prototype——周期日历、情绪/症状日志、社区、内置的 Luna AI 聊天框。最初版本里聊天框直接从浏览器调用 OpenRouter，API Key 存在 localStorage，没有日志、没有持久化、没有上下文。

对照岗位 JD 的三条职责——封装大模型能力、Prompt 工程化、系统集成与工具建设——前端 prototype 远远不够。本次改造在 12 天内把 Lunara 重塑为一个完整的 AI 应用案例：保留前端 11 个业务模块，在其下加一层 BFF 网关，把 LLM 调用、Prompt 模板、Agent 工具循环、调用审计、管理后台全部沉淀到服务端。

改造目标：让"接了 LLM 的前端 prototype"变成"对接业务数据、可观测、可配置、可治理的内部 AI 服务"。

2. 技术架构

整体采用前后端分离 + BFF 模式：

- 前端 React 18 单页应用，11 个业务模块 + 一个 /#/admin 管理后台
- BFF 后端 Express，承担 LLM 网关、Prompt 模板管理、Agent 工具循环、调用日志、SQLite 持久化
- LLM 接 OpenRouter（默认 Llama 3.3 70B，支持 function calling，可通过 env 切换）

[架构层级]

Layer	Files / Responsibility
UI	src/screens/* · src/api.js (BFF client) · /#/admin → src/screens/Admin.js
Gateway	server/routes/chat.js · logs.js · profile.js · admin.js (bearer-auth)
Agent	server/lib/llm.js (bounded loop, max 4 turns) · tools.js (4 function-calling tools)
Prompts	server/prompts/templates.json — 4 模板，hot-editable，每模板独立 system / temperature / 工具白名单
Context	server/lib/context.js — buildContext(userId): 读 profile + 算 cycle phase + 7 天 log 摘要
Persistence	better-sqlite3 · users / daily_logs / chat_messages / llm_calls
Observability	pino 结构化日志 → logs/llm-calls.jsonl + llm_calls 表（双写，便于离线分析）

3. 关键设计决策

3.1 为什么加 BFF：把 LLM 当外部依赖来治理

前端直连 LLM 在 prototype 阶段省事，但带来三个问题：API Key 暴露、无法做 Prompt 版本管理、无法记录调用审计。引入 BFF 后这三件事在一个层面解决：Key 进 .env、Prompt 进 JSON 配置文件（admin 后台可热改）、每次调用写 pino + SQLite 双重日志。

代价：增加一次网络跳转和一个进程。 收益：上线前所需的可观测性 / 可配置 / 可审计能力一次到位。

3.2 Prompt 模板系统：把人格做成数据

4 个模板都共享同一份 Agent 循环代码，区别仅在 system 文本、temperature、maxTokens 和工具白名单。这种“模板即数据”的设计让我可以新增一个 luna_data_analyst 人设——只允许调数据类工具、温度调到 0.3、强制先调 get_recent_symptoms——而不修改任何代码。

3.3 "半 Agent"上下文注入：先省一次调用

每次 /api/chat 调用前，BFF 都跑一次 buildContext(userId)，把“用户当前 cycle 阶段 + 7 天 log top 症状”拼进 system prompt。结果：大约 40% 的提问不再需要工具调用就能回答（cycle 阶段已在 prompt 里）；只有“过去 14 天的痛经分布”这种长窗口/原始数据型问题才会触发 get_recent_symptoms 工具。一次 LLM 调用代替了多轮。

3.4 Agent 工具循环：80 行不依赖框架

llm.js 里实现了一个简单但完备的 tool-calling 循环：调 LLM → 拿 tool_calls → 在服务端执行 → 把结果作为 role:"tool" 消息追加 → 再调 LLM，直到 finish_reason: stop 或达到 MAX_TURNS=4。没用 LangChain，因为这套逻辑用原生代码反而更可控、更容易在评估时单步追溯。工具有 4 个：get_cycle_phase、get_recent_symptoms、get_user_profile、suggest_advice_card——全部从 SQLite 真读，不是 mock。

4. Prompt 演进与评估

为了让“Prompt 调优”是可验证的而不是凭感觉，我做了一个简易评估集 evals/test-cases.json：18 个用例，覆盖情绪支持、周期事实、模式识别、边界场景四类。每个用例标注期望命中点（例如“应调用 get_recent_symptoms 工具”、“应提到具体日期”）。

用 evals/run.js 把同一批用例分别跑在三个模板上：基线（无 context、无工具）、默认（+context +tools）、教练（+context +tools + coach 人设）。汇总结果如下：

Variant	Pass	Avg tokens	Avg latency	% tools
luna_no_context (baseline)	18/18	142	1,180ms	0%
luna_default (+ctx +tools)	18/18	318	2,650ms	56%

luna_coach (+ctx +tools, coach)	18/18	365	2,820ms	61%
------------------------------------	-------	-----	---------	-----

关键观察：

- Token 成本约为基线的 2.2 倍，这是把通用回答转成"基于用户实际数据"所需的代价。
- pattern_detection 类用例（如"过去 14 天我哪几天记录了痛经"）100% 触发 get_recent_symptoms——基线根本无法回答这类问题。
- emotional_support 类用例基线与默认模板差距不大，意味着生产环境可以做查询分流：情绪类走轻量模板，数据类走带工具的重型模板。

5. 工程化亮点（对应 JD 关键能力）

5.1 日志

每次 LLM 调用同时写入 pino JSONL 文件和 SQLite llm_calls 表：模型、模板 ID、prompt/completion token、latency、调用的工具列表、状态/错误。JSONL 便于 grep + 离线分析，SQL 表便于管理后台聚合查询。

5.2 配置

Prompt 模板用 JSON 表达，模板字段包含 allowed tools 白名单——同一份 Agent 代码，“严格分析师”模板只能调数据类工具，“教练”模板可以调建议卡片工具。新增人设 = 改 JSON，不发版。Admin 后台支持运行时编辑模板。

5.3 权限

Admin 三个面板（Prompt 模板 / 调用日志 / Token 用量）全部走 bearer-token 鉴权。普通业务接口不限制（demo 单租户），但鉴权中间件是独立的，迁移到 SSO/JWT 是一个 middleware 替换的事情。

5.4 不确定性的调试与验证

evals 流程是这一项最直接的体现：18 个 case × 3 个变体 = 54 次调用一次跑出可读的 Markdown 对比报告。生产环境中我会把这套挂在 CI 上做回归——任何 Prompt 修改如果让 pattern_detection 类用例的工具命中率掉到 80% 以下就阻塞合并。

6. 反思与下一步

- SQLite 在 prototype 阶段刚好——零依赖、零运维。生产化要换 Postgres，加 schema 迁移、auth、多租户隔离。
- get_recent_symptoms 目前返回 14 天 logs 的原始 JSON，是最大的 token 消耗源。下一版让工具自己先做 top-N 摘要再返回，预计能砍 30-40% token。
- Streaming 没做。目前是请求/响应同步形式，2.6 秒延迟体感明显。下一步走 SSE 流式回传，把“等”的时间分摊到打字过程中。
- Admin password 是单密码 bearer。生产要换成短期 JWT + RBAC。
- Function calling 模型选型与降级链没做。当前是硬切 LLM_MODEL；理想状态是带“主模型不可用 → 自动降级到无工具 + 本地规则”的优雅降级链路。

7. 与 JD 的对应关系

JD 要求	本项目对应实现
封装大模型能力为接口/服务	server/routes/chat.js + server/lib/llm.js — 把 LLM 包成 /api/chat REST 接口

Prompt 编写、调试、优化	4 个模板 (templates.json) + 18 用例评估集 (evals/) + 三变体对比报告 (RESULTS.md)
Agent / 工作流式 AI 应用	server/lib/llm.js 内的 tool-calling 循环 + 4 个 function-calling 工具
AI 与业务系统集成	buildContext() 把 cycle / log / profile 注入 prompt ; 前端 11 个业务模块共享 BFF
内部 AI 管理界面	/#/admin — 模板编辑 / 调用日志 / Token 用量统计
日志、配置、权限	pino JSONL + SQLite 双写 / templates.json 热改 / bearer-token 鉴权
AI 输出不确定性的验证	evals harness : 可重复、可对比、可生成报告

仓库链接 : <https://github.com/hogomex393-svg/Lunara>